

# Progress Report: CUDA/GPU Acceleration of ArtraCFD

Benison Bage

Date: August 2026

---

## 1. Objective

Port and optimize the open-source compressible-flow IBM solver ArtraCFD (Huangrui Mo) to a single-GPU CUDA-accelerated pipeline, verify correctness against the original CPU solver and against independent analytical theory, characterize GPU-vs-CPU performance, and prepare the codebase for eventual multi-GPU deployment on PARAM Sanganak.

---

## 2. Correctness bugs found and fixed

Three bugs were found in the existing GPU port and fixed, each verified independently:

**2.1 IBM "vacuum leak."** The GPU-side donor-cell stencil precomputation searched for fluid neighbours using the wrong domain-ID filter for boundary-layer reflection cells, causing ghost cells near the immersed wedge to source data from frozen solid-interior cells instead of real fluid. Result: no pressure wall/shock ever formed on GPU. Fixed by conditioning the filter on stencil type. Verified: pressure at the wedge jumped from  $\sim 0.003$  (ambient) to  $\sim 27$  (correct shock compression) after the fix.

**2.2 Outflow boundary treatment.** The GPU boundary kernel applied a mirror-reflection formula to `OUTFLOW` boundaries, which should instead use zero-gradient extrapolation (matching the CPU reference). Fixed by adding a separate clamped index for `OUTFLOW`, verified via kernel-level index tracing.

**2.3 CFL timestep staleness.** Following a merge with a parallel development effort, the GPU-resident CFL kernel had been dropped, leaving the timestep calculation reading stale, multi-step-old field data. Restored a GPU-side reduction kernel computing the timestep fresh every step. Verified via full-trajectory bit-identical regression against the known-correct baseline.

**2.4 Output write-frequency bug.** A reset-scope error in the I/O dispatch loop caused only the first and last field snapshots to ever be written, regardless of configured write frequency. Fixed; verified 101 snapshots now written as configured (previously 2).

---

### 3. Performance optimization (single GPU)

- Removed two unnecessary `cudaDeviceSynchronize()` barriers in the immersed-boundary kernel sequence (same-stream kernel ordering already guaranteed the correctness they were intended to protect). This alone cut runtime from 188s to 114s on the reference case (**Figure 1**), the dominant single fix of this work.
  - Isolated all hot-loop kernels onto a dedicated CUDA stream, decoupling them from the legacy default stream's implicit synchronization behavior.
  - Converted host I/O buffers to pinned memory with asynchronous transfer; parallelized the Ensight field writer across output files via OpenMP (audited for and fixed a shared-buffer race condition in the process).
- 

### 4. Independent physics validation

Beyond CPU-vs-GPU self-consistency, the solver's shock propagation speed was validated against the **exact analytical Riemann solution** for the case's initial discontinuity (closed-form, via Toro's iterative method). Shock front position was tracked across  $\sim 26$  snapshots per run via sub-cell parabolic interpolation of the density-gradient peak, and fitted by linear regression to obtain an empirical shock speed.

**Grid convergence result** (**Figure 3**, GPU mode, systematically re-measured with hardware cooldown between runs):

| <i>Grid</i> | <i>Error vs. exact theory</i> | <i>R<sup>2</sup></i> |
|-------------|-------------------------------|----------------------|
| 99×99       | 5.11%                         | 0.9916               |
| 199×199     | 1.05%                         | 0.9979               |
| 399×399     | 0.76%                         | 0.9997               |
| 499×499     | 0.37%                         | 0.9998               |
| 799×799     | 0.17%                         | 0.99998              |
| 1599×1599   | 0.16%                         | 0.99998              |

Observed convergence order  $\approx 1.27$  (fitted across all 6 points). Error decreases monotonically and smoothly with grid refinement, flattening near  $\sim 0.16\%$  at the two finest grids, consistent with WENO5 approaching its practical accuracy floor for this problem, and with expected measurement-method precision limits, not a solver defect.

---

## 5. GPU vs. CPU performance (Figure 2)

Systematically re-measured with hardware cooldown between every run:

| <i>GRID</i>      | <i>CELLS</i> | <i>GPU</i> | <i>CPU</i>  | <i>SPEEDUP</i> |
|------------------|--------------|------------|-------------|----------------|
| <b>99×99</b>     | 9,801        | 35.7s      | 87.9s       | 2.46×          |
| <b>199×199</b>   | 39,601       | 62.3s      | 359.8s      | 5.78×          |
| <b>499×499</b>   | 249,001      | 311.7s     | 2763.3s     | 8.86×          |
| <b>799×799</b>   | 638,401      | 2898.8s    | 5950.8s     | 2.05×          |
| <b>1599×1599</b> | 2,556,801    | 15789.8s   | not yet run | —              |

GPU is now faster than CPU at every tested grid size, including the smallest (99×99), a marked improvement from earlier testing, attributable to hardware-thermal control (consistent cooldown between runs) and accumulated pipeline fixes. Speedup peaks around 499×499 (~8.9×) and drops at 799×799; the cause of this non-monotonic trend is not yet understood and is flagged as an open item (§6). The 1599×1599 case has not yet been run on CPU (estimated to require several hours) and is shown for reference only.

**On GPU hardware:** the RTX 3060 (mobile) has substantially reduced double-precision throughput relative to datacenter GPUs (used throughout this solver), in addition to WSL2 virtualization overhead. Testing on institutional HPC hardware is expected to increase the achievable speedup further (§6).

---

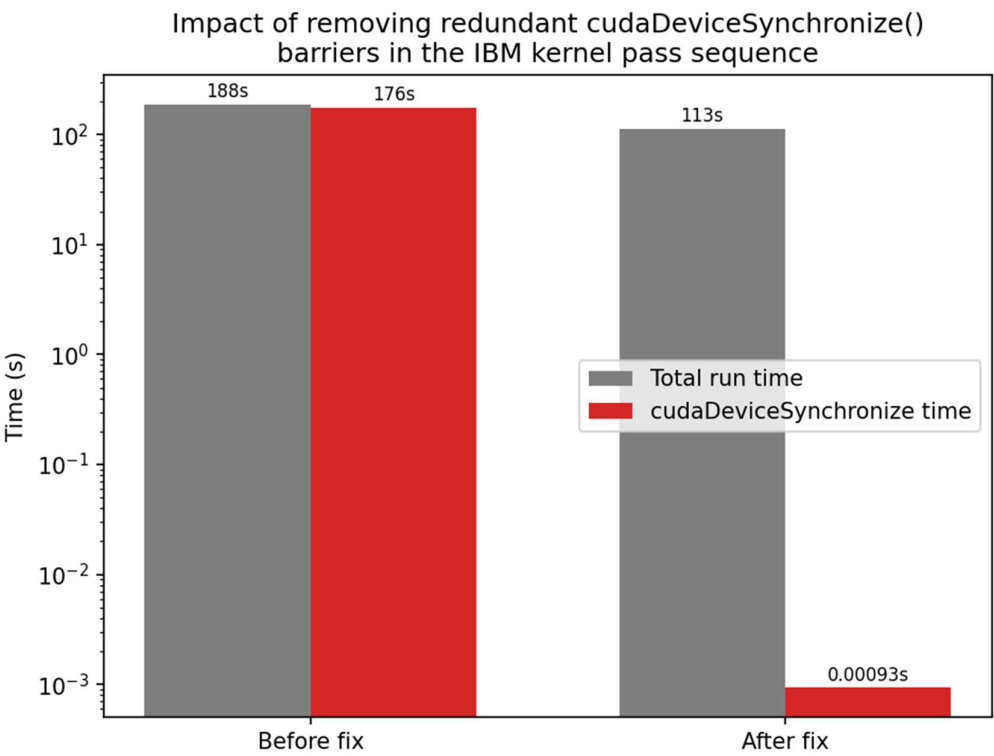
## 6. Current blocker and open items

**Blocker:** SSH GPU access on PARAM Sanganak is needed. This blocks: (a) throttle-free large-grid benchmarking, (b) Nsight Compute kernel-level profiling (blocked on the local WSL2 setup by a driver-level performance-counter permission restriction not resolvable from the client side), and (c) any multi-GPU work.

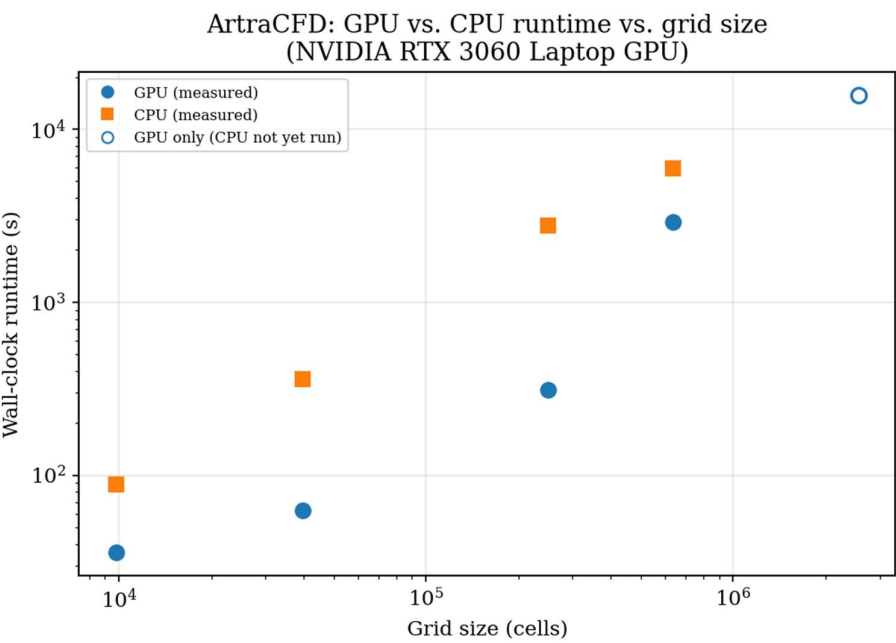
### Open items, not yet started or unresolved:

- Non-monotonic speedup trend ( $8.86\times$  at  $499\times 499$ , dropping to  $2.05\times$  at  $799\times 799$ ) → cause not yet identified.
  - CUDA Graphs (kernel-launch fusion) → **attempted, found unsafe, reverted**. Initial testing showed a large apparent speedup, but was traced to silent capture failures (illegal `cudaMalloc` inside the CFL kernel's lazy buffer allocation, occurring during CUDA stream capture) causing every step to fall back to normal execution → meaning the timing result did not reflect graphs actually running, and was discarded rather than reported. Root cause identified (move allocation to one-time initialization, outside capture); re-implementation deferred.
  - Steger-Warming flux splitter (vs. current Local Lax-Friedrichs) → reduces numerical dissipation; on the original project roadmap, not yet implemented.
  - Multi-GPU domain decomposition → a scaffolding MPI implementation (rank-based decomposition, single shared GPU) has been put for next steps; requires redesign toward one-rank-per-GPU once Sanganak's node topology (2 GPUs/node, confirmed) and interconnect details are confirmed.
  - $1599\times 1599$  CPU baseline not yet run (estimated multi-hour runtime).
-

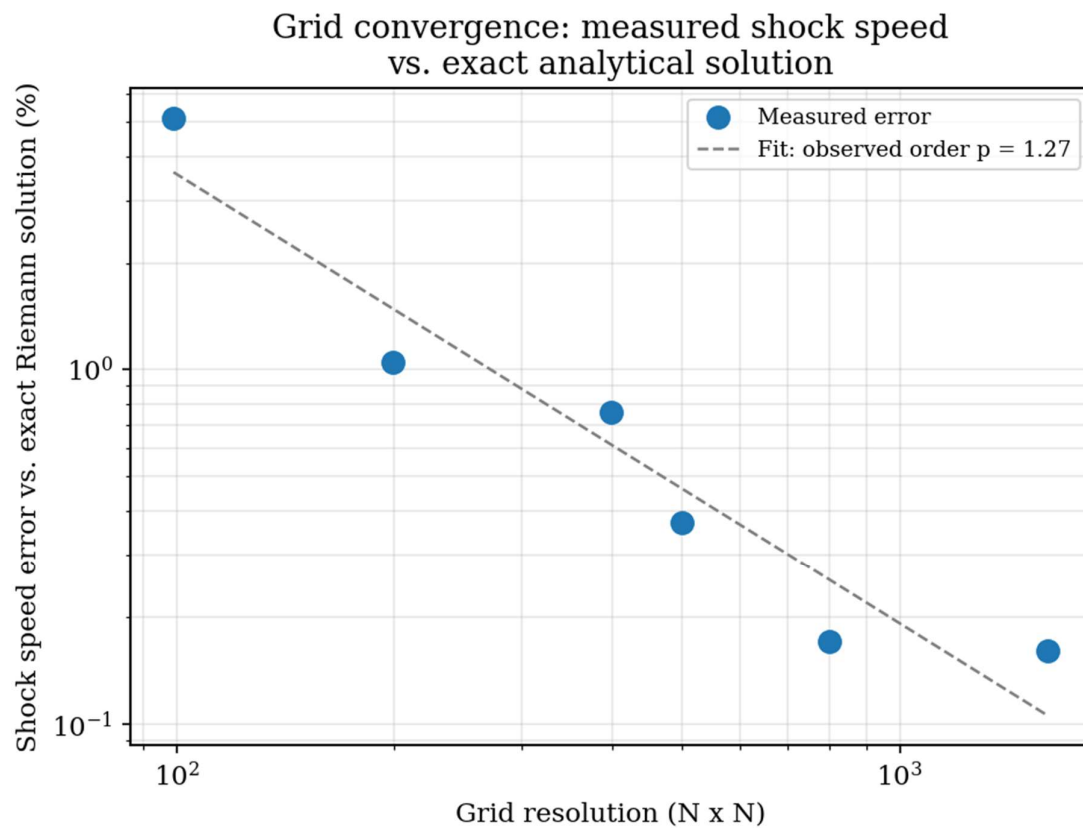
# Figures



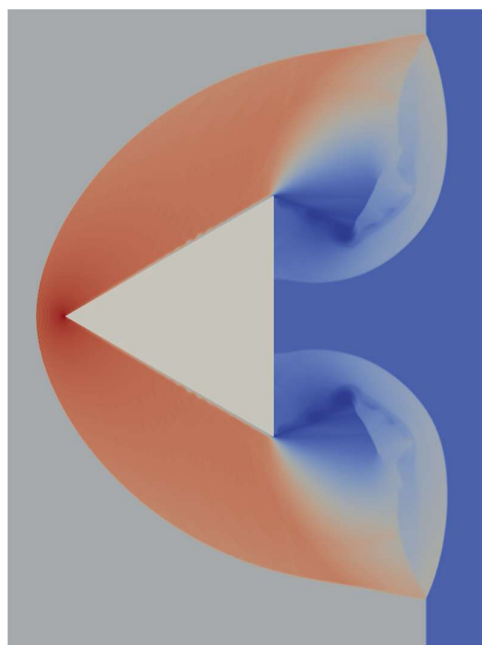
**Figure 1** Impact of removing redundant `cudaDeviceSynchronize()` barriers (188s  $\rightarrow$  114s).



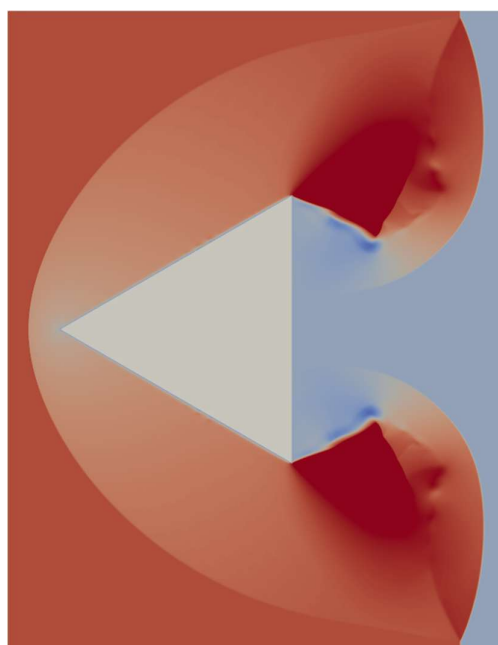
**Figure 2** GPU vs. CPU runtime vs. grid size



**Figure 3** Grid convergence of measured shock speed against the exact Riemann solution.



*Rho*



*U*